COMPANION NOTE · PREPARED FOR THE PLATFORM TEAM · DATA ENGINEER

Different market, **same job.**

Why Lifepal's regulated insurance marketplace maps onto Abound's consumer lending.

In Lifepal, the other side was the insurers; at Abound, it's the loan itself. Lifepal connected people with the insurers that cover them, and priced and checked risk on the way; Abound connects people's bank data, through Open Banking, to a lending decision it makes and funds. In both, a transactional database is the system of record, data changes after the fact, a customer can ask to be forgotten, and a real financial decision depends on the number being right. The data problems that follow are the ones NEXUS was built to solve.

Aurora · DMS · Glue · Spark · Iceberg · Athena

BUILT IN lifepal → PRESENTED TO **Abound**

ONE FLOW, TWO BUSINESSES

Abound side from public material

01 Apply
Lifepal · a customer asks for a quote
Abound · a customer applies for a loan

→

02 Share
Lifepal · personal and policy data, with consent
Abound · Open Banking transactions, with consent

→

03 Decide
Lifepal · risk and fraud models, then the insurer
Abound · affordability and the scorecard

→

04 Change
Lifepal · claims, renewals, late statuses
Abound · repayments, arrears, re-categorised rows

THE CLOSEST PARALLEL

risk decisions on data that keeps changing

LIFEPAL · PERSON → INSURER

Customer	Data	Model	Outcome
Needs cover; applies once.	Policy, quote and claim rows in PostgreSQL and MySQL.	Customer-risk and claims-fraud models (LightGBM, XGBoost).	Claims and renewals arrive for months.

ABOUND · PERSON → LOAN

Customer	Data	Model	Outcome
Applies for a loan; connects their bank.	Applications, loans, repayments in Aurora; Open Banking rows.	A scorecard on transaction data, calibrated by band.	Repayments, misses and cures, every month.

THE SAME PROBLEM	HOW I HANDLED IT AT LIFEPAL	WHERE IT SHOWS HERE
Analytics off the production DB	Moved reporting, ad-hoc analytics and model training onto the S3 lake	The role's first 6–12 months; gold tables for each workload
Changes twice, late, out of order	Merged on the source log position, idempotent under replay	change_seq ordering; a full replay changed 0 rows
Features as they were known	Point-in-time training sets; training/serving parity	PIT income differs for 72 of 111 decisions
Risk and selection bias	Causal inference with data scientists, to separate real drivers from selection	Calibration by band, on approved loans only, stated as such

Described at the level I can share publicly; I'm happy to walk through the details in conversation.

WHAT ABOUND RUNS

as the role lists it · data here is synthetic

SOURCE AND CAPTURE
Aurora PostgreSQL · Aurora MySQL · AWS DMS · CDC

Here: a simulated lender (customers, applications, loans, repayments, Open Banking rows) and the files DMS would write, with six planted problems.

PROCESS AND STORE
Spark · AWS Glue · Iceberg · S3

Here: PySpark on Iceberg 1.7, as Glue 5.0 runs it; bronze, silver with history, gold.

SERVE AND RUN
Athena · Lambda · ECS · Fargate · Python

Here: layouts planned for Athena cost; Lambda and Fargate around the job in Terraform.

SCALE
£1.6bn
lending capacity after a £250m Deutsche Bank facility in March 2025, as reported.

CONCEPT MAP

same grain, different names

LIFEPAL · INSURANCE MARKETPLACE	ABOUND · CONSUMER LENDING (MY READING)	GRAIN	SHARED DATA PROBLEM
Policy and quote rows PostgreSQL · MySQL	**Applications, loans, repayments** Aurora	one row per source record	Get every change out, exactly once
Claim status updates late, for months	**Repayment outcomes** missed · paid late	one row per change	Newest change wins; history kept
Risk at quote time as-of features	**Risk at decision time** as-of decided_at	one row per decision	No feature from after the decision
Risk and fraud models LightGBM · XGBoost	**The scorecard** calibrated by band	score band × cohort	Predicted against observed
Reporting and partners insurer packs	**Reporting and funders** loan book · facility tape	month end	One number for every audience
Erasure requests GDPR	**Erasure requests** GDPR	one person, every table	Deleted everywhere, including snapshots

Lifepal examples are from my own work. Abound examples come from the role, public information, and the synthetic data in this project.

WHAT CARRIES OVER DIRECTLY

proven at Lifepal

CDC into the lake
Lifepal: DMS and Debezium from PostgreSQL and MySQL; reporting latency 24 h → under 5 min.
Here: DMS files keyed on change_seq; 0 rows changed by a full replay.

Off the production DB
Lifepal: Reporting, analytics and training moved onto the lake; ad-hoc requests −60%.
Here: Gold tables for the loan book, calibration and features.

Layout and cost
Lifepal: Nightly batch 6 h → 90 min; $11k a month against a $19k projection.
Here: 90% less scanned per customer read on Iceberg.

Data Science partner
Lifepal: Point-in-time training sets, parity, drift and recalibration.
Here: Calibration by score band; PIT income feature.

Governance
Lifepal: 500+ checks in CI, PII masking, RBAC, lineage; escalations −60%.
Here: Reconciliation caught 16 rows; erasure across snapshots.

STATED PLAINLY

what's real

Real: the Lifepal outcomes, from my CV and production work, and all the code in the repository, which runs end to end in CI.

Synthetic: every customer, loan, repayment and transaction. The Abound side comes from the job description and public information; no Abound systems or data were used.

Depth: DMS, Spark, Glue, Aurora/RDS PostgreSQL and MySQL, Lambda and ECS are production experience. Delta is my production table format; this repository is my Iceberg work. Athena is working level.

QUESTIONS I'D BRING

for the Platform team

1. Which workloads still read Aurora directly, and which would Data Science move first?
2. How are DMS gaps and restarts detected today: heartbeat, volume, or reconciliation?
3. Where do model calibration and its features live now, and how fresh must they be?
4. Which AWS line items are growing fastest: Glue, Athena, S3 or DMS?
5. How do erasure requests reach the lake today?